

When an AI grades its own work, which grades does it use?

A small measurement, and one result I did not expect.

Jake Tiller · 20 August 2026 · Research note, not peer reviewed. One person designed the tests, ran them, and graded the results. No second reviewer checked a single case. That is the biggest limit on everything here.

The setup

A lot of new AI tooling works like this. The model produces an answer, then attaches a label to its own answer saying how good it is. Verified or unverified. Pass, fail, or needs review. High confidence or low. The label is supposed to let you audit the output without reading all of it.

That only works if the model actually uses the labels. So I checked which ones it uses.

The test

I gave a model problems that answer themselves. One was a deploy script that fails on Windows because it hardcodes forward slashes. The cause is stated in the problem. There is nothing left to discover.

Then I asked it to analyse the problem and label each piece of its own analysis, including with a label that means *this added nothing*.

On a problem that contains its own answer, that label is correct for every piece. It was never used once.

What came back

Four experiments, two different labelling systems, 118 labelled decisions.

WHAT WAS MEASURED	RESULT
Share taken by the two most-used labels	75 to 100%
Share taken by the strongest "this is wrong" labels	0 to 2%
Times one label fired across 22 chances	0
Times two more fired across 96 chances	0

The model was not lying and it was not broken. Every label it chose was defensible. It simply kept choosing the ones that were easiest to defend.

The part I did not expect

Making one label harder to use did not push the model toward the correct label. It pushed the model to the next-easiest label instead.

I tried to fix the problem the obvious way. I wrote a strict test for each label, so you could not use one without showing your work. Usage of the loosest label dropped from seven of eight cases to one of eight, which looked like a win.

But the traffic did not move to the label I was trying to promote. That one went from one case to two. A different label, one I had not touched, went from zero cases to four.

So the fix did not remove the problem. It relocated it. If that holds up, it means patching a single label in a labelling system is close to useless, because whatever you squeeze simply reappears next door.

A second thing worth knowing

One of the systems first splits a paragraph into separate claims, then labels each claim. I ran the same paragraph through it three times.

It produced 14 claims, then 8, then 10.

Any design that stores those labelled claims in a shared database, so it can link and compare them later, needs the claims to line up between runs. When the same paragraph splits into 14 pieces one time and 8 the next, there is nothing stable to line up.

What this does not show. The sample is small: 6, 8, 8 and 96 labelled decisions. These are not rates for any population.

One model family was used throughout, so this may be a fact about that family rather than about self-grading in general.

I wrote the tests, chose the problems, planted the defects, predicted the outcomes and graded them. Nobody else labelled a single case. **That is the largest weakness here and I have not fixed it.**

One of the three changes I made between rounds cannot be isolated, because I changed three things at once. The shift in labels is measured. Which change caused it is not.

Four ways to prove me wrong

Each is a procedure you can finish in one sitting. None needs anything from me.

- **Two model families, same problems.** Take any labelling scheme with a "this adds nothing" option. Give it three problems that state their own answers, three runs each, on two different model families. Count how often that label fires per family. If the rates differ, this is about the model, not about self-grading.
- **Label ten cases yourself.** Take ten items the model already labelled, hide its labels, and assign your own using the same written definitions. Compare. If your labels bunch into the same two options, the definitions are the problem. If they spread out, the model is.
- **Three variants, one change each.** I changed three things between rounds: I tightened the loosest label, I reordered which label gets considered first, and I required the rejecting label to be checked before the others. Build three versions that each change only one. Run the same problems. Whichever moves the labels is the cause.
- **Swap one judgement for a computation.** Replace the rejecting label's test with something a script can decide. For a metaphor label, strip the metaphor words from the output and check whether what is left restates the input. Count the fire rate before and after. That is the control condition I never ran.

If you want to try the cheap version

Take whatever labelling scheme you already use. Give it something that states its own answer, so the "nothing here" label is the correct one. Then count how often it says so.

That is the whole experiment. It takes about ten minutes and it tells you whether your labels are controls or decoration.

The full note, with raw counts per experiment, the predictions written before each run, and the exact test wording, is public in the Resilience-Ledger repository on GitHub, under [research/label-absorption-note](#). Corrections welcome, particularly from anyone who runs one of the four tests and gets a different answer.